Control Density Functions for Whole-Arm Obstacle Avoidance and Trajectory Tracking

Author Names
Department and Institution
City, Country
Email addresses

***Abstract*—These notes present a control density function quadratic program for whole-arm obstacle avoidance and Cartesian trajectory tracking. A navigation density combines smooth obstacle factors with a regularized target weighting. Parameterizing the density-weighted control flux produces an affine transport constraint and a convex quadratic program with joint-velocity bounds. The pointwise optimization admits a reduction from 168 flux coefficients to at most seven coordinates. The formulation is evaluated on a six-joint UF850 manipulator with arm and gripper collision geometry. A tracking experiment reaches a final position error of 4.84 mm while retaining positive recorded obstacle clearance. The optimized implementation achieves a median nominal-controller and filter time of 1.034 ms in a CPU replay benchmark. Additional rollouts at 50, 100, and 240 Hz retain positive clearance without measured computation-budget overruns. These results establish finite-run kinematic performance; physical robot execution and closed-loop safety guarantees remain to be validated.**

***Index Terms*—Control density functions, quadratic programming, robot manipulators, obstacle avoidance, trajectory tracking.**

I. Introduction

Main Contributions

This work develops a density-based command filter for robot manipulator motion in cluttered environments. The main contributions are:

1) **Finite-dimensional CDF-QP and computational reduction.** A quadratic weighted-flux parameterization yields one aggregate density transport constraint alongside joint-rate bounds. The pointwise problem is reduced from 168 coefficients to at most seven coordinates without changing its optimum.
2) **Whole-arm trajectory tracking.** Smooth factors for all robot-sphere–obstacle-sphere pairs are combined into one density. Arm and gripper geometry thereby influence the filtering of a nominal Cartesian pose-tracking command.
3) **Simulation and timing evaluation.** UF850 experiments assess tracking, obstacle clearance, intervention, and numerical feasibility. Optimized CPU benchmarks and multiple update rates assess computation time relative to the available step budgets.

The central modeling choice is to combine all robot–obstacle sphere pairs in one obstacle factor. The resulting QP contains a single scalar density transport inequality, rather than an inequality for each pair. The number of pairs still affects geometry and derivative evaluation.

The formulation is evaluated as a pointwise command filter. The reported results demonstrate empirical obstacle avoidance and tracking; they do not establish sampled-data safety of the recomputed controller.

II. Preliminaries and Density Construction

A. Navigation Problem

Consider the control-affine system

$$\dot{x} = f(x) + g(x)u, \tag{1}$$

where $x \in \mathcal{X} \subseteq \mathbb{R}^n$, $u \in \mathcal{U} \subseteq \mathbb{R}^m$, and the vector fields are continuously differentiable. Let $\mathcal{X}_0$, $\mathcal{X}_T$, and $\mathcal{X}_u$ denote the initial, target, and unsafe sets.

The navigation objective is to reach $\mathcal{X}_T$ from $\mathcal{X}_0$ while avoiding $\mathcal{X}_u$ and respecting the input constraints. In trajectory tracking, the objective is to follow a prescribed reference while maintaining obstacle clearance.

Density-based navigation encodes target and unsafe regions through a scalar field. The construction is motivated by an occupancy interpretation: desirable trajectories should avoid occupying the unsafe region and approach the target [1], [2]. A suitable condition on density transport is also required; the scalar field alone is not a certificate.

B. Smooth Obstacle Factors

Let the unsafe set be

$$\mathcal{X}_u = \bigcup_{\ell=1}^{N_c} \{x : d_\ell(x) \le 0\}, \tag{2}$$

where N_c counts modeled collision conditions. Positive signed clearance d_ℓ indicates separation, zero indicates contact, and negative clearance indicates penetration.

For a sensing width $\delta_\ell > 0$, define

$$S(s) = \begin{cases} 0, & s \le 0, \\ \dfrac{\exp(-1/s)}{\exp(-1/s) + \exp[-1/(1-s)]}, & 0 < s < 1, \\ 1, & s \ge 1. \end{cases} \tag{3}$$

The inverse obstacle bump is

$$\Psi_\ell(x) = S\left(\frac{d_\ell(x)}{\delta_\ell}\right). \tag{4}$$

It vanishes on the unsafe region, transitions through the sensing region, and equals one beyond that region.

Combining the factors gives

$$\Psi(x) = \prod_{\ell=1}^{N_c} \Psi_\ell(x). \tag{5}$$

Thus, $\Psi = 0$ whenever any modeled collision condition is active. Outside all sensing regions, $\Psi = 1$. For an obstacle-free scene, the product is defined as one.

C. Target Weighting

Let $V(x)$ be positive away from the target. A navigation density takes the form

$$\rho(x) = \frac{\Psi(x)}{V(x)^\alpha}, \qquad \alpha > 0. \tag{6}$$

The numerator encodes obstacle geometry and the denominator weights the target region [1], [2].

The robot implementation uses

$$V_\epsilon(q) = \|p_{\mathrm{TCP}}(q) - p_g\|^2 + \epsilon, \qquad \epsilon > 0, \tag{7}$$

where p_{TCP} is the tool-center-point position and p_g is the final Cartesian goal. The implemented density is

$$\rho(q) = \frac{\Psi(q)}{V_\epsilon(q)^\alpha}. \tag{8}$$

Regularization keeps the density finite at the goal. Theoretical results requiring a singular target density therefore need additional justification for this implementation.

III. Robot Model and Nominal Controller

A. Euler–Lagrange Equations

For a fully actuated robot with n_q joints, let $q \in \mathbb{R}^{n_q}$ denote joint position and $v = \dot{q}$ joint velocity. The kinetic energy and Lagrangian are

$$\mathcal{T}(q, v) = \frac{1}{2} v^\mathsf{T} M(q) v, \qquad L = \mathcal{T} - P(q). \tag{9}$$

The Euler–Lagrange equations give

$$\frac{\mathrm{d}}{\mathrm{d}t}\frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = \tau - F(\dot{q}). \tag{10}$$

Equivalently,

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + F(\dot{q}) = \tau, \tag{11}$$

where $M(q) = M(q)^\mathsf{T} \succ 0$ is inertia, $C(q, \dot{q})\dot{q}$ contains Coriolis and centrifugal terms, $G(q) = \nabla_q P(q)$ is gravity, and F represents friction.

With

$$x = \begin{bmatrix} q \\ v \end{bmatrix}, \qquad h(q, v) = C(q, v)v + G(q) + F(v), \tag{12}$$

the control-affine representation is

$$\dot{x} = \underbrace{\begin{bmatrix} v \\ -M(q)^{-1}h(q, v) \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ M(q)^{-1} \end{bmatrix}}_{g(x)} \tau. \tag{13}$$

B. Velocity-Level Implementation

The evaluated controller uses

$$\dot{q} = u, \qquad q, u \in \mathbb{R}^6, \tag{14}$$

with componentwise bounds

$$-u_{\max}\mathbf{1} \le u \le u_{\max}\mathbf{1}. \tag{15}$$

Here u is a joint-velocity command.

The simulation integrates

$$q_{k+1} = q_k + \Delta t\, u_k \tag{16}$$

and assigns the resulting joint positions directly to the PyBullet robot. A physical implementation would require an inner velocity servo that tracks the command accurately.

> **Evaluated model**
>
> The Euler–Lagrange equations describe the physical arm. The experiments evaluate joint-velocity integration, collision geometry, and computation time. Motor dynamics and hardware tracking are not evaluated.

C. Nominal Pose-Tracking Controller

The nominal controller is OSCBF's `PoseTaskVelocityController` [4]. It tracks TCP position and orientation without considering obstacles. The CDF-QP subsequently filters its joint-velocity command.

1) Reference: The TCP follows a straight Cartesian segment with minimum-jerk timing:

$$p_{\mathrm{ref}}(t) = p_0 + s(\sigma)(p_g - p_0), \tag{17}$$

where $p_0 = p_{\mathrm{TCP}}(q_0)$, $\sigma = \mathrm{clip}(t/T_f, 0, 1)$, and

$$s(\sigma) = 10\sigma^3 - 15\sigma^4 + 6\sigma^5. \tag{18}$$

For $0 \le t \le T_f$,

$$\dot{p}_{\mathrm{ref}}(t) = \frac{30\sigma^2 - 60\sigma^3 + 30\sigma^4}{T_f}(p_g - p_0). \tag{19}$$

The duration is $T_f = 7\,\mathrm{s}$. Reference velocity and acceleration vanish at both endpoints. The desired tool orientation is fixed:

$$R_d = R(q_0). \tag{20}$$

2) Position and Orientation Feedback: The linear velocity command is

$$v_{\mathrm{cmd}} = \dot{p}_{\mathrm{ref}} + K_p\big(p_{\mathrm{ref}} - p_{\mathrm{TCP}}(q)\big). \tag{21}$$

Let $R = [r_1, r_2, r_3]$ and $R_d = [r_{d,1}, r_{d,2}, r_{d,3}]$ map tool-frame coordinates to the world frame. The controller's orientation-error vector is

$$\mathbf{e}_R = -\frac{1}{2}\sum_{i=1}^{3} r_i \times r_{d,i}. \tag{22}$$

This vector is expressed in the world frame, consistently with the angular Jacobian. With zero desired angular velocity,

$$\omega_{\mathrm{cmd}} = -K_R \mathbf{e}_R. \tag{23}$$

The gains are $K_p = 1.4$ and $K_R = 1.0$.

3) Joint-Velocity Mapping: Let $J(q) \in \mathbb{R}^{6\times 6}$ be the world-frame TCP geometric Jacobian, ordered as linear velocity followed by angular velocity. The mass-weighted generalized inverse is

$$J^{\#} = M^{-1}J^{\mathsf{T}}\left(JM^{-1}J^{\mathsf{T}}\right)^{-1}. \tag{24}$$

The nominal command is

$$\begin{aligned} \widehat{u}_{\mathrm{nom}} &= J^{\#}\begin{bmatrix} v_{\mathrm{cmd}} \\ \omega_{\mathrm{cmd}} \end{bmatrix}, \\ u_{\mathrm{nom}} &= \mathrm{clip}\left(\widehat{u}_{\mathrm{nom}}, -u_{\max}\mathbf{1}, u_{\max}\mathbf{1}\right). \end{aligned} \tag{25}$$

Clipping is componentwise, with $u_{\max} = 0.7$ rad/s.

The mass-weighted branch uses the direct inverse in (24), without a damping term. It therefore requires a nonsingular task inertia matrix; clipping alone does not address singularity handling.

The controller applies a null-space posture task only when the number of joints exceeds the six-dimensional pose-task dimension. Consequently, the configured posture gain contributes no additional command for this six-joint experiment.

Roles of the two controllers

The nominal controller supplies Cartesian tracking and orientation regulation. The CDF-QP minimizes deviation from that command while imposing the pointwise density transport condition and joint-rate bounds. Tracking is an objective, rather than a separate hard constraint.

IV. CDF-QP and Finite-Dimensional Approximation

A. Density Transport

For a differentiable feedback field, define

$$R(x) = \nabla_x \cdot (\rho(x)[f(x) + g(x)u(x)]). \tag{26}$$

CDF-based navigation conditions constrain this residual over an operating domain under the assumptions of the associated theorem [1], [2]. A positive numerical margin is written as

$$R(x) \geq \eta, \qquad \eta > 0. \tag{27}$$

The divergence contains spatial derivatives of the feedback, motivating a parameterized feedback field. Introduce

$$w(x) = \rho(x)u(x), \qquad d_\rho(x) = \nabla_x \cdot (\rho(x)f(x)). \tag{28}$$

With $g = [g_1, \ldots, g_m]$,

$$\begin{aligned} R(x) &= d_\rho(x) \\ &\quad + \sum_{j=1}^{m}\left[g_j(x)^{\mathsf{T}}\nabla_x w_j(x) + w_j(x)\nabla_x \cdot g_j(x)\right]. \end{aligned} \tag{29}$$

For the Euler–Lagrange model, $\nabla_x \cdot g_j = 0$, since M depends only on q. With torque-weighted flux $w = \rho\tau$,

$$R(q, v) = d_\rho(q, v) + \sum_{j=1}^{n_q} e_j^{\mathsf{T}} M(q)^{-1}\nabla_v w_j(q, v). \tag{30}$$

This identity describes the torque-level formulation.

For the implemented velocity model, $f = 0$ and $g = I_6$, giving

$$R(q) = \nabla_q \cdot w(q). \tag{31}$$

B. Quadratic Flux Basis

Set $z = q/\pi$ and collect monomials through total degree two:

$$\Phi_2(q) = \mathrm{col}\left(1, \{z_i\}_{i=1}^{6}, \{z_i z_j\}_{1\leq i\leq j\leq 6}\right) \in \mathbb{R}^{28}. \tag{32}$$

There are one constant, six linear, and twenty-one quadratic terms.

Define

$$b(q) = \Psi(q)\Phi_2(q), \qquad w_j(q;\theta_j) = b(q)^{\mathsf{T}}\theta_j, \tag{33}$$

where $\theta_j \in \mathbb{R}^{28}$. Stacking all channels gives

$$\theta = \mathrm{col}(\theta_1, \ldots, \theta_6) \in \mathbb{R}^{168}. \tag{34}$$

Where $\rho > 0$, the recovered input is

$$u_j(q;\theta_j) = \frac{w_j(q;\theta_j)}{\rho(q)} = V_\epsilon(q)^\alpha \Phi_2(q)^{\mathsf{T}}\theta_j. \tag{35}$$

The obstacle factor cancels in the quotient, but remains in the transport constraint and influences the optimized coefficients.

C. Command and Transport Maps

For numerical evaluation, define

$$\bar{\rho}(q) = \max\{\rho(q), 10^{-10}\}, \tag{36}$$

and

$$B(q) = I_6 \otimes \frac{b(q)^{\mathsf{T}}}{\bar{\rho}(q)} \in \mathbb{R}^{6\times 168}. \tag{37}$$

The command is $u_\theta = B\theta$.

Define the transport row

$$a(q)^{\mathsf{T}} = \left[\left(\frac{\partial b}{\partial q_1}\right)^{\mathsf{T}} \cdots \left(\frac{\partial b}{\partial q_6}\right)^{\mathsf{T}}\right]. \tag{38}$$

Then

$$R_\theta(q) = \sum_{j=1}^{6}\frac{\partial w_j}{\partial q_j} = a(q)^{\mathsf{T}}\theta. \tag{39}$$

All derivatives hold θ fixed.

D. Finite-Dimensional QP

At the current configuration, solve

$$\begin{aligned} \theta^\star = \arg\min_{\theta} \quad & \frac{1}{2}\left\|B\theta - u_{\mathrm{nom}}\right\|^2 \\ & + \frac{\lambda_u}{2}\left\|B\theta\right\|^2 + \frac{\lambda_\theta}{2}\left\|\theta\right\|^2 \\ \text{subject to} \quad & a^{\mathsf{T}}\theta \geq \eta, \\ & -u_{\max}\mathbf{1} \leq B\theta \leq u_{\max}\mathbf{1}. \end{aligned} \tag{40}$$

The filtered command is $u^\star = B\theta^\star$.

The objective preserves the nominal command, penalizes command magnitude, and regularizes the coefficients. Its Hessian and linear term are

$$H = (1 + \lambda_u)B^{\mathsf{T}}B + \lambda_\theta I, \tag{41}$$

$$c = -B^{\mathsf{T}}u_{\mathrm{nom}}. \tag{42}$$

For $\lambda_\theta > 0$, H is positive definite, so a feasible QP has a unique optimizer.

There are 168 coefficients and 13 scalar inequalities: one aggregate transport condition and twelve joint-rate bounds.

E. Exact Pointwise Reduction

The command and constraint depend on θ through six input values and one residual. Define

$$A(q) = \begin{bmatrix} B(q) \\ a(q)^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{7\times 168}. \tag{43}$$

Let $Q \in \mathbb{R}^{168\times r}$ have orthonormal columns spanning $\mathrm{range}(A^{\mathsf{T}})$, with $r \le 7$. Any coefficient vector decomposes as

$$\theta = Q\xi + \theta_{\perp}, \qquad A\theta_{\perp} = 0. \tag{44}$$

The component $\theta_{\perp}$ changes neither the command nor the residual, but increases $\|\theta\|^2$. Therefore, $\theta_{\perp} = 0$ at the optimum.

Setting

$$\widetilde{B} = BQ, \qquad \widetilde{a} = Q^{\mathsf{T}} a \tag{45}$$

gives

$$\begin{aligned} \xi^{\star} = \arg\min_{\xi\in\mathbb{R}^r} \quad & \frac{1}{2}\left\|\widetilde{B}\xi - u_{\mathrm{nom}}\right\|^2 \\ & + \frac{\lambda_u}{2}\left\|\widetilde{B}\xi\right\|^2 + \frac{\lambda_\theta}{2}\|\xi\|^2 \\ \text{subject to} \quad & \widetilde{a}^{\mathsf{T}}\xi \ge \eta, \\ & -u_{\max}\mathbf{1} \le \widetilde{B}\xi \le u_{\max}\mathbf{1}. \end{aligned} \tag{46}$$

This reduction is exact for the stated pointwise objective and constraints. It is performed after evaluating the frozen-coefficient transport map.

The optimized implementation uses a seven-coordinate QP and fused JAX evaluation. On the tested replay samples, its maximum command difference from the full formulation is 1.11×10^{-5} rad/s, below the specified 10^{-4} rad/s tolerance.

> **Computational structure**
>
> The flux field has 168 coefficients, but the pointwise optimization needs only six command values and one transport residual. Coefficient directions that affect neither quantity can be removed, leaving at most seven optimization coordinates.

F. Scope of the Pointwise Condition

The residual in (39) is evaluated with frozen coefficients. The online controller recomputes $\theta^{\star}(q,t)$, whose spatial derivatives are absent from that expression. The constraint is also checked only at sampled states.

The measured residual therefore does not establish a domain-wide inequality or sampled-data invariance for the recomputed policy. The experiments report empirical tracking, clearance, and numerical feasibility.

The QP has no slack and no general feasibility guarantee. The density floor changes $w = \rho u$ wherever it is active. Self-collision and joint-position constraints are not included.

V. UF850 Setup and Implementation

A. Whole-Arm Geometry

Robot sphere a has center $p_a(q)$ and radius r_a. Obstacle b has center o_b and radius R_b. Their surface clearance is

$$d_{ab}(q) = \|p_a(q) - o_b\| - r_a - R_b. \tag{47}$$

TABLE I
Robot, scene, and controller setup.

Setting	Value
Robot	UF850, six joints
Gripper	Fixed closed configuration
Robot collision spheres	20
Obstacle spheres	5
Robot–obstacle pairs	100
Aggregate transport inequalities	1
Reference duration T_f	7 s
TCP offset	0.172 m
Position/orientation gains	1.4/1.0
Joint-rate bound $u_{\max}$	0.7 rad/s
Sensing width δ	0.10 m
Density exponent α	0.2
Target regularization ϵ	10^{-3}
Transport margin η	0.01
Input penalty λ_u	0.02
Coefficient penalty λ_θ	2×10^{-4}
Optimized rollout rates	50, 100, 240 Hz

The obstacle factor is

$$\Psi(q) = \prod_{a=1}^{N_r}\prod_{b=1}^{N_o} S\left(\frac{d_{ab}(q)}{\delta}\right). \tag{48}$$

The closed-gripper model contains 20 collision spheres: 13 arm/wrist spheres and seven additional gripper-cover spheres attached to the wrist frame. With five obstacle spheres, 100 pairwise factors enter (48), while the QP retains one aggregate transport inequality.

Robot sphere centers are transformed from their local link coordinates using differentiable forward kinematics. The gripper cover is specific to the fixed closed configuration. The collision-model implementation provides the local sphere centers and radii [5].

Independent PyBullet mesh-distance and obstacle-contact queries provide a second geometric measurement. Sphere and mesh clearances need not agree because they use different robot representations.

B. Scene and Parameters

The experiment uses a seven-second Cartesian reference through five static obstacles. The initial joint configuration is

$$q_0 = [0, -0.7, -1.0, 0, 0.8, 0]^{\mathsf{T}} \text{ rad}, \tag{49}$$

and the final goal is

$$p_g = [0.38, 0.35, 0.43]^{\mathsf{T}} \text{ m}. \tag{50}$$

The gripper remains closed, and the TCP is offset 0.172 m along the wrist tool axis.

Table I summarizes the setup. Obstacle centers and radii are listed in Appendix A.

The moving reference enters through u_{nom}. The density denominator retains the fixed final goal. Thus, this is a static-density filter with a time-varying nominal command, rather than the time-varying density construction in [3].

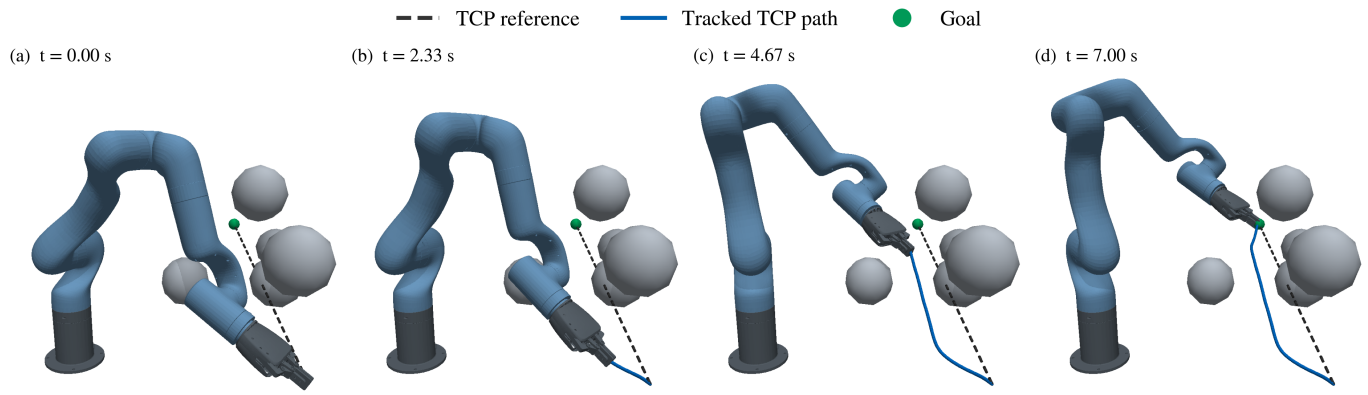


Fig. 1. UF850 motion in the optimized 100 Hz rollout at $t = 0$, 2.33, 4.67, and 7.00 s. The dashed curve is the TCP reference, the blue curve is the accumulated TCP path, and the green marker denotes the goal. The gripper remains closed. Snapshots are rendered from saved joint states.

Algorithm 1 Online Density-Based Joint-Velocity Filter

Require: Kinematics, collision geometry, reference, QP parameters, and timestep Δt

Initialize geometry and the cached solver
Compile and warm up differentiable functions
for each sample k **do**
  Obtain q_k and reference values at t_k
  Compute $u_{\mathrm{nom}}(q_k, t_k)$
  Evaluate clearances, Ψ, ρ, and basis derivatives
  Assemble command and transport maps
  Solve (40) or its reduction (46)
  if the solve is unsuccessful **then**
    Flag failure and terminate the experiment
  end if
  Recover the filtered command u_k
  Integrate $q_{k+1} = q_k + \Delta t\, u_k$
  Update simulated joint positions
  Record tracking, clearance, residual, and timing
end for

VI. Simulation Results

A. Evaluation Protocol

Two related evaluations are reported:

1) **Tracking experiment:** a seven-second trajectory sampled at 240 Hz with 1681 state samples and 1680 commands.
2) **Optimized implementation benchmark:** CPU replay timing and additional rollouts at 50, 100, and 240 Hz.

These correspond to repository validations 9 and 10, respectively.

The coordinate, error, command, and intervention plots describe the tracking experiment. The robot snapshots show the optimized 100 Hz rollout. Timing plots describe the optimized implementation.

B. Robot Motion Through Clutter

Figure 1 shows the robot raising and reconfiguring its arm while approaching the goal. The TCP path departs from the straight reference during the middle of the motion and approaches the goal in the final snapshot.

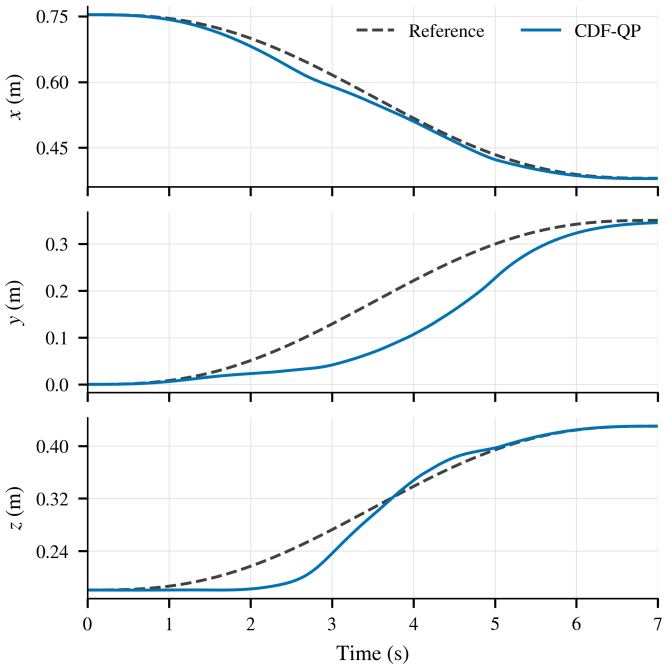


Fig. 2. TCP coordinates in the tracking experiment. Dashed curves denote the reference and solid curves denote the CDF-QP response. Time is simulated time.

The optimized 100 Hz rollout ends with a 4.76 mm position error. Its minimum sphere and mesh clearances are 20.41 mm and 48.49 mm, with zero recorded obstacle-contact steps. These measurements complement the snapshots, which show only four selected instants.

C. Cartesian Tracking

Figure 2 compares the TCP coordinates with the moving reference in the tracking experiment. The largest visible deviation occurs in y during the middle of the motion. The z coordinate initially remains below its reference and subsequently rises toward the final height.

Position error is

$$e_p(t) = \|p_{\mathrm{TCP}}(q(t)) - p_{\mathrm{ref}}(t)\|\,. \tag{51}$$

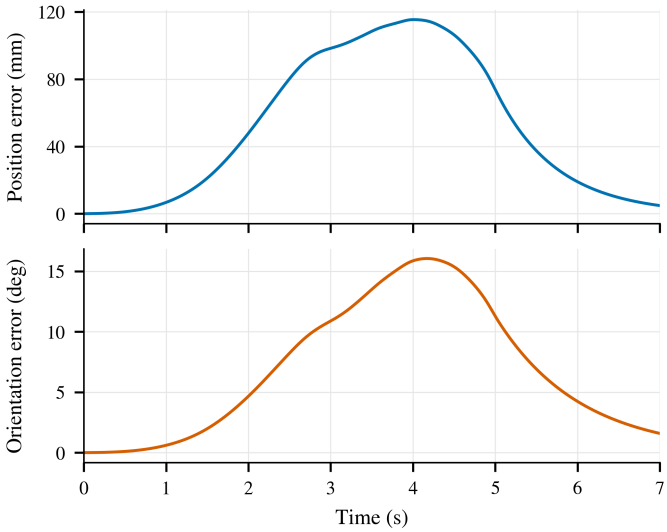


Fig. 3. Position error relative to the moving reference and orientation angle relative to the fixed desired tool orientation. Time is simulated time.

TABLE II
TRACKING ERRORS. RMS VALUES USE ALL 1681 UNIFORMLY SPACED STATE SAMPLES.

Error	RMS	Maximum	Final
Position (mm)	66.82	115.37	4.84
Orientation (deg)	8.80	16.06	1.59

The reported orientation error is the rotation angle

$$e_{\mathrm{ang}}(t) = \cos^{-1}\left[\mathrm{clip}\left(\frac{\mathrm{tr}(R_d^{\mathsf{T}} R(q(t))) - 1}{2}, -1, 1\right)\right]. \quad (52)$$

This scalar angle differs from the feedback vector $\mathbf{e}_R$ in (22).

Figure 3 shows errors increasing during the middle of the motion and decreasing near the final reference. The position RMS and maximum errors are $66.82\,\mathrm{mm}$ and $115.37\,\mathrm{mm}$. The orientation RMS and maximum errors are $8.80°$ and $16.06°$.

The final errors decrease to $4.84\,\mathrm{mm}$ and $1.59°$. Thus, the experiment demonstrates small terminal errors after a substantial transient tracking deviation.

D. Joint Commands and Intervention

Figure 4 compares nominal and filtered joint velocities. The filter modifies several joints simultaneously, particularly joints 1, 2, 3, and 5 during the middle of the motion. The maximum absolute command is $0.684\,\mathrm{rad/s}$, below the $0.7\,\mathrm{rad/s}$ bound.

Define intervention as

$$I_k = \|u_k - u_{\mathrm{nom},k}\|_2. \quad (53)$$

Its RMS and maximum are $0.178\,\mathrm{rad/s}$ and $0.328\,\mathrm{rad/s}$. The zero-order-hold integral is

$$\Delta t \sum_k I_k = 0.876\ \mathrm{rad}. \quad (54)$$

The lower panel of Fig. 5 shows intervention concentrated in the middle of the motion, overlapping the tracking-error transient. Intervention includes the effects of regularization and the transport constraint; it is not a safety-constraint activation measure or a causal comparison with an unfiltered rollout.

TABLE III
CLEARANCE, FEASIBILITY, AND COMMAND MODIFICATION IN THE TRACKING EXPERIMENT.

Metric	Value
Minimum sphere clearance	20.37 mm
Minimum mesh clearance	48.54 mm
Obstacle-contact steps	0
Minimum transport residual	0.0100
Maximum joint speed	0.684 rad/s
RMS intervention	0.178 rad/s
Maximum intervention	0.328 rad/s
Integrated intervention	0.876 rad
All QPs feasible	Yes

TABLE IV
OPTIMIZED CPU REPLAY TIMING IN MILLISECONDS.

Component	Median	p95	Maximum
Nominal + filter	1.034	1.091	1.562
Density filter	0.757	0.811	1.287
Solver only	0.057	0.094	0.108

E. Clearance and Transport Residual

Both recorded clearance measures remain positive in Fig. 5. Their minima are $20.37\,\mathrm{mm}$ for the collision spheres and $48.54\,\mathrm{mm}$ for the PyBullet mesh queries. No obstacle-contact steps are reported.

The residual remains near $\eta = 0.01$ over much of the trajectory and rises near the end. No margin violations are recorded using a 10^{-8} tolerance, and all QPs are reported feasible.

Tracking and clearance

The tracking experiment reaches a final position error of $4.84\,\mathrm{mm}$ while retaining positive recorded clearances and zero obstacle contacts. The larger RMS and maximum errors quantify the transient departure from the reference.

F. Optimized CPU Timing

The optimized implementation was benchmarked on an Intel Core i7-10700K CPU at $3.80\,\mathrm{GHz}$ using JAX 0.4.30 and Clarabel 0.11.1, with one requested BLAS thread. The benchmark evaluates 160 saved states over three repeats, giving 480 timed evaluations after explicit warm-up.

Figure 6 and Table IV summarize the results. The median nominal-controller and filter time is $1.034\,\mathrm{ms}$, the 95th percentile is $1.091\,\mathrm{ms}$, and the maximum is $1.562\,\mathrm{ms}$. The filter median is $0.757\,\mathrm{ms}$, while the solver-call median is $0.057\,\mathrm{ms}$.

These timing scopes are nested: the filter includes the solver, and nominal plus filter includes the filter. They must not be added together.

Nominal-plus-filter timing includes reference generation, nominal control, JAX evaluation and host synchronization,

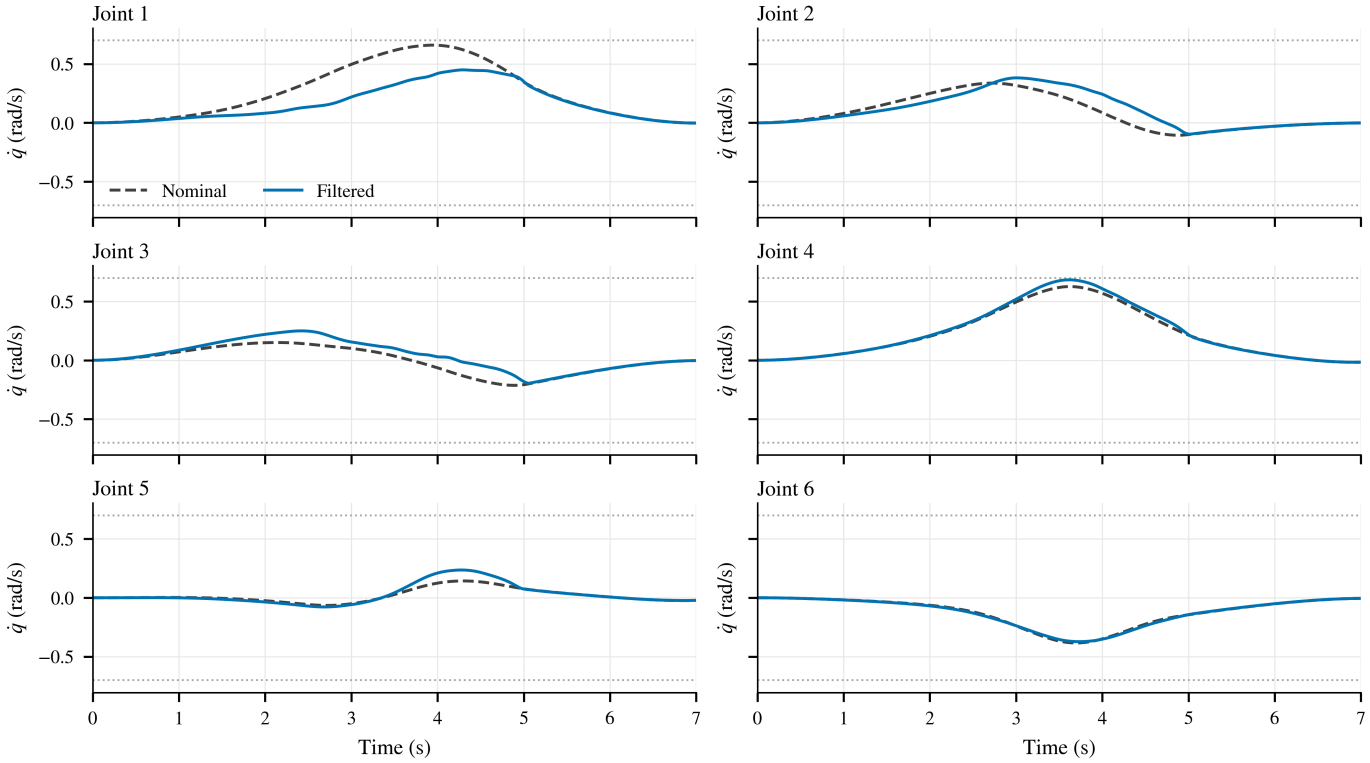


Fig. 4. Nominal and filtered joint velocities. Dashed curves denote nominal commands, solid curves denote filtered commands, and dotted lines mark the ± 0.7 rad/s bounds. A command at t_k acts over the following integration interval.

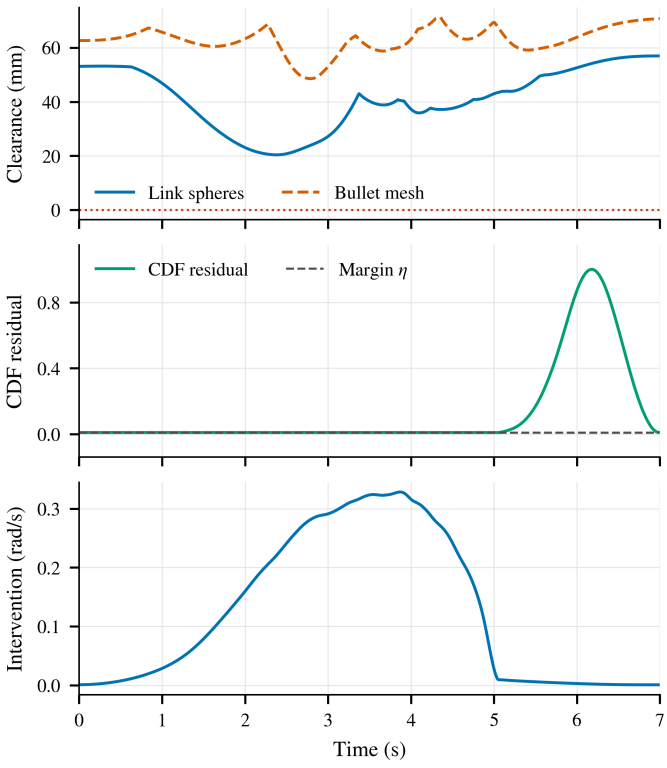


Fig. 5. Clearance, transport residual, and intervention. Top: minimum sphere and mesh clearances. Middle: raw CDF residual and margin $\eta = 0.01$. Bottom: $\|u - u_{\text{nom}}\|_2$. Clearance is evaluated after integration; residual and intervention are evaluated at the command state before integration.

TABLE V
FIXED-RATE COMPUTATION TIMES. BUDGETS AND DURATIONS ARE IN MILLISECONDS.

Rate	Budget	Ctrl. p95	Ctrl. max	Loop max
50 Hz	20.000	1.083	1.405	2.250
100 Hz	10.000	1.096	1.409	2.245
240 Hz	4.167	1.131	1.873	3.373

matrix construction, checks, QP update and solve, and filter-return diagnostics. It excludes Bullet stepping, rendering, sensor input, and communication.

G. Fixed-Rate Rollouts

Additional rollouts use simulated update rates of 50, 100, and 240 Hz, corresponding to step budgets of 20, 10, and 4.167 ms. Each follows the seven-second reference.

Figure 7 normalizes computation time by the associated budget. The controller timing includes nominal control and the filter. The diagnostic-loop timing additionally includes joint integration, Bullet stepping, mesh and contact queries, and logged diagnostics.

At 100 Hz, the maximum controller and loop times are 1.409 ms and 2.245 ms. At 240 Hz, they are 1.873 ms and 3.373 ms, approximately 45% and 81% of the 4.167 ms budget. No measured controller or diagnostic-loop budget overruns occur in these runs.

All runs reach the target, all QPs solve successfully, and no obstacle contacts or density-floor activations are recorded.

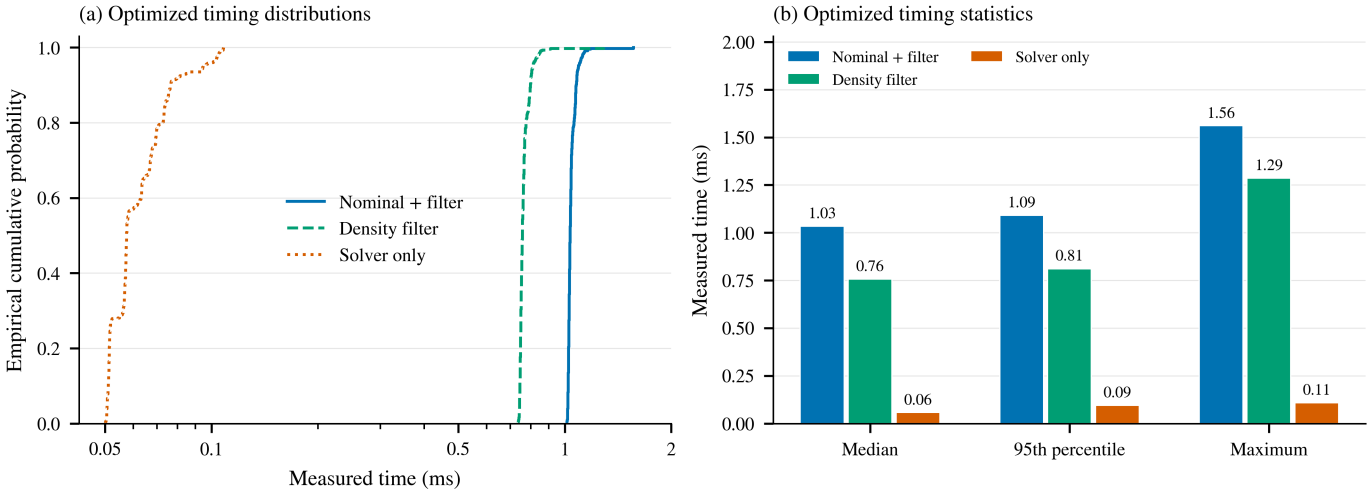


Fig. 6. Optimized CPU replay timing over 480 evaluations after warm-up. Left: empirical cumulative distributions on a logarithmic time axis. Right: median, 95th-percentile, and maximum durations. Timing scopes are nested, not additive. Bars summarize measured samples rather than confidence intervals across independent experiments.

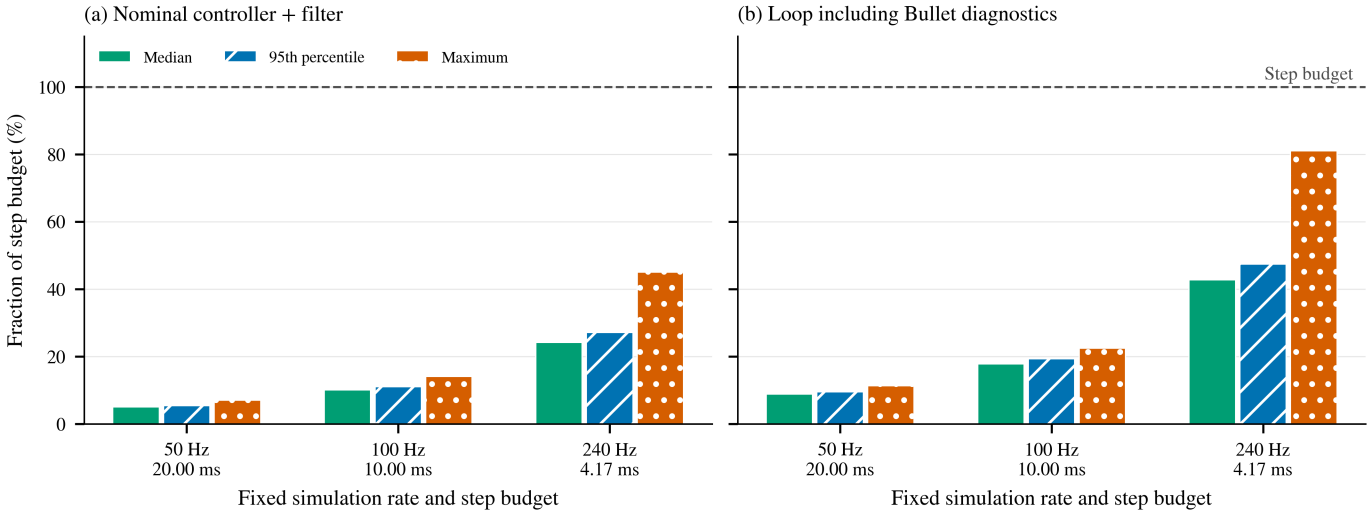


Fig. 7. Measured computation time as a percentage of the simulated step budget. Left: nominal controller and filter. Right: the loop including Bullet stepping and diagnostics. All measured maxima remain below the dashed 100% budget line. The rollouts are headless and unpaced.

TABLE VI
MINIMUM CLEARANCE AND TERMINAL ERROR ACROSS UPDATE RATES. ALL DISTANCES ARE IN MILLIMETERS.

Rate	Sphere	Mesh	Final error
50 Hz	20.49	48.40	4.62
100 Hz	20.41	48.49	4.76
240 Hz	20.37	48.54	4.84

Minimum clearances and terminal errors remain similar across update rates (Table VI). The maximum joint speed remains below 0.7 rad/s at every tested rate.

Optimized implementation

The median nominal-plus-filter replay time is 1.034 ms. All tested update rates retain positive clearance without measured budget overruns. The 100 Hz run uses at most 2.245 ms of its 10 ms budget when diagnostics are included.

H. Interpretation and Remaining Validation

The figures provide complementary evidence: snapshots show robot configuration, tracking plots quantify reference deviation, command plots show joint-rate limits, and clearance queries assess the modeled collision geometry. Timing experiments assess computation separately from tracking accuracy.

The rollouts are headless, unpaced kinematic experiments. Measured durations below simulated step budgets demonstrate computational headroom for these finite runs, not hard real-time bounds. Rendering, sensing, communication, scheduling jitter, and physical motor tracking remain outside the reported controller timing.

The experiments also use one scene and reference. They do

not establish robustness across initial conditions or obstacle placements, general sphere coverage, self-collision avoidance, or superiority over a matched CBF baseline.

VII. Conclusion

A density-based command filter was formulated using smooth obstacle factors and a quadratic weighted-flux basis. The QP combines all modeled robot–obstacle pairs in one aggregate transport condition alongside joint-rate bounds. Its 168-coefficient pointwise formulation admits reduction to at most seven coordinates.

UF850 simulations demonstrate tracking through clutter with positive recorded clearance and zero obstacle contacts. The optimized implementation achieves millisecond-scale CPU updates and remains within the measured step budgets at 50, 100, and 240 Hz. Further work is required to analyze the recomputed sampled controller and validate geometry coverage and physical execution.

Appendix A
Obstacle Configuration

Table VII lists the five static obstacle spheres in the world frame. Robot collision-sphere centers and radii are specified by the implementation's `make_uf850_collision_model` function [5].

TABLE VII
World-frame obstacle centers and radii. All values are in meters.

Obstacle	x	y	z	Radius
1	0.5665	0.2553	0.3056	0.075
2	0.4900	0.2844	0.5415	0.080
3	0.2924	0.2063	0.2649	0.076
4	0.6790	0.0892	0.4238	0.084
5	0.5021	0.3967	0.3615	0.061

References

[1] J. Moyalan, S. S. K. S. Narayanan, A. Zheng, and U. Vaidya, "Synthesizing controller for safe navigation using control density function," arXiv:2403.14464, 2024.

[2] J. Moyalan, S. S. K. S. Narayanan, and U. Vaidya, "Control density function for robust safety and convergence," arXiv:2407.05133, 2024.

[3] S. S. K. S. Narayanan, J. Moyalan, and U. Vaidya, "Safe navigation in dynamic environments using density functions," arXiv:2411.12206, 2024.

[4] D. Morton and M. Pavone, "Safe, task-consistent manipulation with operational space control barrier functions," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025, pp. 187–194.

[5] S. Krishnamoorthy, "xarm-density: UF850 density-based control implementation." [Online]. Available: https://github.com/sriram-2502/xarm-density